

In-Game Chat Toxicity Intent Classification

Cameron Letendre

Abstract

Inside this paper we explore the possibility of creating a more accurate prediction of toxicity in gaming messages, while trying to preserve speed for a real-time overlay. Modern gaming is known to be very structurally different from common language, but more than that because of the heavy involvement of slang, acronyms, and in-game vocabulary along with an average of 4 characters per message this creates a challenge for machine learning models. In this paper we attempt to do intent classification on these messages predicting them into one of four groups. (E) Explicit Toxicity, (I) Implicit Toxicity, (A) Action, and (O) Other. This classification task for this assignment is done by a distilbert model that is fine tuned on a labeled dataset known as CONDA.

Keywords: Mixed-Precision, Iterative Refinement, Conjugate Gradient, Sparse Linear Systems, Preconditioning.

Introduction

In modern computing today, Floating-point Systems control how a computer's ability to represent real numbers impacts both the speed and accuracy of scientific calculations. A major challenge in this field is to solve Systems of Linear Algebraic Equations (SLAE), following the form:

$$\mathbf{Ax} = \mathbf{b}$$

where $\mathbf{A} \in \mathbb{R}^{n \times n}$ is the coefficient matrix, $\mathbf{x} \in \mathbb{R}^n$ is the unknown solution vector, and $\mathbf{b} \in \mathbb{R}^n$ is the known right-hand side vector. Solving a SLAE means determining the vector $\mathbf{x}$, typically using direct methods such as Gaussian elimination or LU Decomposition, or through iterative techniques.¹

SLAE are present everywhere, used to simulate real-world physics applied in many places like modeling fluid dynamics, analyzing structural integrity, and finding the best-fit line for a dataset using the Least Squares method. As the scale of these simulations increases, the size of the systems grows, becoming incredibly large and complex. This high computational cost means that solving SLAE can easily create a bottleneck in High Performance Computing (HPC). So optimizing the performance of linear solvers is a critical objective in numerical analysis.

Floating-Point Standards

In the **IEEE Standard for Floating-Point Arithmetic**,² computers handle numbers consistently using two specific formats:

- **(Float64) Double Precision:**
The standard for high accuracy. It uses 64-bit memory to provide high precision with 53 bits for significant digits. With the large precision though it results in slower memory bandwidth usage.
- **(Float32) Single Precision:**
The standard for high speed. It uses 32-bit memory to take up half of the space and moves faster than double precision, but is less accurate because of a smaller range of numbers.

1. **elman2014finite**.

2. **IEEE754_2019**.

Even though both of these methods have their own strengths, and weaknesses, traditionally scientific simulations utilize double precision floating-point arithmetic to ensure high accuracy and stability. Researchers though, are always looking for ways to create an optimum system for both speed and accuracy, and this is where the idea of **Mixed Precision Iterative refinement** solvers come in.³

The idea is that you can use a lower-precision inner solver to do the heavier computational load for the bulk of the work, while having a high-precision outer loop to ensure the final answer is correct.

Because sparse iterative solvers are typically memory bound rather than compute bound, explained by the Roofline Model⁴ and confirmed on modern hardware,⁵ performing the inner solve in single precision reduces the volume of data transferred through memory, improving the overall amount being passed. This paper goes over this with partial differential equations that do not change over time called elliptic equations. In this paper we are interested specifically in the Pressure Poisson Equation that occurs in incompressible turbulent flow simulations.⁶

1 The Mixed Precision Iterative Refinement (IR) Algorithm

1.1 Iterative Refinement

The concept of Iterative Refinement (IR), was originally developed to improve the accuracy of algorithms that calculate the exact solution of a SLAE in a finite number of steps, known as a direct solver.⁷ This has been adapted though to the mixed-precision paradigm to effectively and efficiently solve large SLAE.⁸ This approach leverages the speed of lower-precision arithmetic while maintaining the stability and final accuracy with higher precision arithmetic.

As we said before the Mixed Precision IR algorithm operates two floating-point environments, an outer loop high-precision format and an inner loop lower-precision format. The core idea is to perform the bulk of the expensive matrix operations in the faster, lower precision loop, and only use the higher precision for the update steps to mitigate the building of rounding errors from the lower precision.

The IR method proceeds iteratively to refine an approximate solution vector $\mathbf{x}^{(i)}$ using the following key steps:

1. **Residual Calculation (High Precision):** Compute the residual vector $\mathbf{r}^{(i)}$ using double precision (Float64). This step quantifies the discrepancy between the current approximation and the true solution without losing information:

$$\mathbf{r}^{(i)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(i)}$$

2. **Correction System Solution (Low Precision):** The residual $\mathbf{r}^{(i)}$ is casted to single precision (Float32) for the error correction equation. The system is solved using a single-precision copy of the matrix $\mathbf{A}$, denoted as $\mathbf{A}_{sp}$:

$$\mathbf{A}_{sp}\mathbf{z}_{sp}^{(i)} \approx \mathbf{r}_{sp}^{(i)}$$

Here, $\mathbf{z}_{sp}^{(i)}$ represents the error correction vector computed in single precision. This step is where the majority of the computational work occurs (the inner CG solve), using the 32-bit functionality to reduce the amount of memory bandwidth that's being used per inner iteration.

3. Higham_Mary_2022.

4. williams2009roofline.

5. Walden2021Performance.

6. chorin1968numerical.

7. wilkinson1964rounding.

8. Medvedev2021.

3. **Solution Update (High Precision):** The single-precision correction vector we get from the step 2 ($\mathbf{z}_{sp}^{(i)}$) is cast back to double precision ($\mathbf{z}^{(i)}$) and applied to the current solution. This update is performed in Float64 to ensure the improvement is applied accurately:⁹

$$\mathbf{x}^{(i+1)} = \mathbf{x}^{(i)} + \mathbf{z}^{(i)}$$

This cycle repeats until a certain level of convergence and accuracy is achieved. The success of the method is dependent on the fact that the matrix-vector multiplication required for the correction system (Step 2) is performed in the fast, lower precision, which is where the bulk of the computational work lies.

1.2 Convergence Termination

The efficiency gain achieved by Mixed Precision IR depends heavily on balancing the performance of the inner solver against the number of outer IR iterations required for convergence. Since the inner solver (Step 2) is the primary source of computational speedup, it must be used efficiently.

Wasting computational cycles by over-converging the inner correction system ($\mathbf{A}\mathbf{z}^{(i)} \approx \mathbf{r}^{(i)}$) negates the speedup gained by using mixed precision. Conversely, under-converging the inner loop can cause the outer IR loop to require too many iterations or even fail to converge.

Therefore, a crucial design choice is the **inner solver's termination criterion**. The objective is to find a correction vector $\mathbf{z}$ that reduces the residual $\mathbf{r}$ just enough to ensure rapid convergence of the outer IR process, without achieving full machine precision in the inner loop. This trade-off is visualized in Figure ??, where an optimal tolerance minimizes the total solution time.

A widely adopted strategy is to use a **relative stopping criterion** based on the initial residual $\mathbf{r}^{(i)}$. The inner solver is typically terminated when the norm of the inner residual satisfies:¹⁰

$$\|\mathbf{r}^{(i)} - \mathbf{A}\mathbf{z}^{(i)}\| \leq \tau \cdot \|\mathbf{r}^{(i)}\|$$

The symbol τ represents the tolerance parameter. It is of the initial residual that is allowed to remain, dictating how much required precision of the inner solution there must be before the inner solver is allowed to stop, and pass the back to the outer loop.

The value τ is often set close to the unit round-off of the lower-precision format (e.g., $\tau \approx 10^{-7}$ for Single Precision). However, empirical tuning (as seen in Figure ??) often reveals that a looser tolerance (e.g., 10^{-3}) can further enhance efficiency by reducing the workload of the inner solver while maintaining a sufficient accuracy for the outer loop update.

2 Experimental Methodology and Solver Configurations

Following the theoretical framework of the Mixed Precision IR procedure, the next critical step is to validate its performance gains and the impact of the inner solver's termination criteria (τ) through experimentation.

The objective of the experimental phase is to first, monitor the practical speedup achieved by the MPIR algorithm compared to a standard Double Precision solver. (a single precision solver test does not work because our relative residual is of 10^{-12} , outside of the range of single precision) Second, to determine the optimal balance point, as visualized in Figure ??, by systematically testing various configurations.

2.1 Test Matrices and Problem Domains

The systems of linear equations used for testing were selected to represent realistic HPC problems that would typically be solves with a SLAE. A primary focus is on matrices derived from the

9. quinlan.

10. Carson2018.

discretization of the Pressure Poisson Equation (PPE), which arises naturally in incompressible turbulent flow simulations, such as those governed by the Navier-Stokes equations. The PPE typically yields large, sparse, and ill-conditioned matrices, making it a highly relevant benchmark for iterative solvers.

In the tests though we also used matrices from structural engineering to ensure the robustness of the mixed-precision approach across different physical domains, as well as cover a variety of ill-conditioned and ranging nonzero sparse matrices. All selected matrices are Symmetric Positive Definite (SPD), which was a necessary condition for effective application of the Conjugate Gradient method.¹¹

2.2 Tools and Environment

All numerical experiments were conducted on a workstation within the University of Southern Maine Computer Science lab. It is equipped with an **11th Gen Intel(R) Core(TM) i7-11700** processor running at **2.50GHz** and **32 GB** of system memory.

The algorithms were implemented in **Julia (version 1.11.7)** running on **Ubuntu 24.04.3 LTS**. We perform sparse matrix operations using Julia’s **SparseArrays** library. Standard IEEE 754 floating-point arithmetic was strictly adhered to for both single (FP32) and double (FP64) precision calculations. Additionally, **Python (version 3.12.7)** was used for running the system pipeline and generating visualization graphs present.

2.2.1 Test Dataset

To evaluate robustness across different scales, a suite of sparse matrices representing the Pressure Poisson Equation was selected from the **SuiteSparse Matrix Collection**.¹² These matrices vary in dimension ($n \times n$) and sparsity, providing a comprehensive test for our system. The matrices were selected based on being Symmetrical Positive determinant (SPD) with varying conditioning and a focus on Computational Fluid Dynamics Problems, but also includes a large 2D/3D engineering.

The properties of the matrices are detailed in Table ?? below.

Matrix Name	Rows (N)	Non-zeros (nnz)	Condition Num. (Est.)
cf1	70,656	1,825,580	2.3×10^3
cf2	123,440	3,085,406	6.8×10^3
consph	83,334	6,010,480	7.7×10^4

Table 1: Properties of the test matrices used in the benchmarks. Condition numbers are estimated using the 1-norm estimation.

2.2.2 Verification Strategy

To precisely measure the accuracy of the solvers, we utilized the Method of Manufactured Solutions. For each test matrix, a true solution vector $\mathbf{x}_{true}$ was defined as a vector of ones:

$$\mathbf{x}_{true} = [1, 1, \dots, 1]^T$$

The right-hand side vector was then computed as $\mathbf{b} = \mathbf{A}\mathbf{x}_{true}$. This allows for the calculation of the explicit relative error

$$\frac{\|\mathbf{x}_{approx} - \mathbf{x}_{true}\|}{\|\mathbf{x}_{true}\|}$$

in addition to the standard residual norm, providing our metric for solver correctness.

¹¹. [hestenes1952methods](#).

¹². [Davis_2011](#).

2.3 Solver Configurations

Conducting the performance comparison, the following three distinct solver configurations we discussed earlier are implemented and benchmarked on our architecture:

1. **Baseline Double Precision:** The standard solver, where all computations are performed entirely in Double Precision. This serves as the reference for accuracy and time consumption, and acts as our baseline for performance. (Implemented as `run_baseline_double` in the codebase).
2. **Mixed Precision (Fixed τ):** The Iterative Refinement algorithm where the outer loop uses *double precision* for the residual calculation and update, and the inner solver uses *single precision*. The inner solver is terminated using a fixed tolerance ($\tau = 10^{-7}$) to isolate the performance cost of over-convergence.
3. **Mixed Precision (Tuned τ):** The Iterative Refinement algorithm using 64/32 bit precision loops, but with the inner termination tolerance τ experimentally tuned (e.g., $\tau = 10^{-3}$) to minimize the total run time, while reflecting the optimal balance point. This configuration corresponds to the `run_tau_sweep` routine in the software implementation.

The performance metric recorded for comparison is the total solution time (Program Run Time) required to achieve an outer residual tolerance of 10^{-12} , ensuring all configurations yield a final solution of equivalently high quality.

2.4 Preconditioning and Stability Mechanisms

To ensure the robustness and speed of the inner solver, two critical numerical techniques were implemented:

2.4.1 Preconditioning Strategy

To accelerate the convergence of the inner Conjugate Gradient solver, we apply a preconditioning step that uses a **single-precision** preconditioner matrix $\mathbf{M} \approx \mathbf{A}_{sp}$.

First the solver will attempt to construct an Incomplete Cholesky factorization with a drop tolerance of 10^{-3} to preserve sparsity. If the factorization is unstable or fails, the system automatically falls back to a Jacobi preconditioner. This hybrid approach ensures that the inner solver remains robust even when the matrix is ill-conditioned. This implementation first involves filtering the matrix for values under a drop tolerance, and then calling `cholesky()` within the Julia standard library. The Jacobi is implemented with extracting the diagonal with `diag()` and computing the inverse with $(1.0./d)$ element wise.

2.4.2 Inner Residual Replacement

Because the inner solver operates in single precision, the recursively updated residual vector tends to drift from the true residual $(\mathbf{b} - \mathbf{Ax})$ due to accumulated rounding errors. To mitigate this, we use a **Residual Replacement** strategy. Every 200 inner iterations, the residual is recomputed from the matrix-vector product to prevent "stagnation" of the convergence, ensuring the inner solver reports an accurate residual norm to the outer loop.

3 Performance Evaluation and Results

This section we'll present the results of the experiment, analyzing the overall run time and speedup achieved by the MPIR algorithm relative to the double-precision baseline. We also look into the results of the sensitivity of the solution time to the inner solver tolerance (τ).

3.1 Overall Performance Comparison

Table ?? summarizes the performance of the three solver configurations across the test matrices. The Tuned results reflect the best time achieved during the τ sweep.

From what we can see from the result, the data demonstrates significant computational gains, in Figure ?. The **Mixed Precision (Fixed)** configuration achieved a speedup between $2.1\times$ and $6.0\times$ compared to the baseline. The **Mixed Precision (Tuned)** configuration even further improved the performance, achieving a speedup as high as $8.6\times$ for the **cfid2** matrix.

Crucially, these performance gains were achieved without compromising solution quality. As shown in Figure ?, the relative error of the Mixed Precision solver remains on the order of 10^{-10} to 10^{-11} , matching the accuracy of the standard Double Precision baseline. Minor variations in the final error are expected due to the different convergence paths taken by the solvers, but both methods successfully satisfy the strict outer residual tolerance of 10^{-12} .

Matrix	Baseline	MP (Fixed $\tau = 10^{-7}$)		MP (Tuned Best)	
	Time (s)	Time (s)	Speedup	Time (s)	Speedup
cfid1	4.11	1.88	2.19 $\times$	0.97 ($\tau = 10^{-2}$)	4.24 $\times$
cfid2	23.10	3.83	6.03 $\times$	2.67 ($\tau = 10^{-4}$)	8.65 $\times$
consph	79.87	33.20	2.41 $\times$	21.04 ($\tau = 10^{-2}$)	3.80 $\times$

Table 2: Overall performance comparison. Speedup is calculated relative to the Baseline Double Precision time. The MP Tuned column indicates the specific τ value that came up with the fastest time.

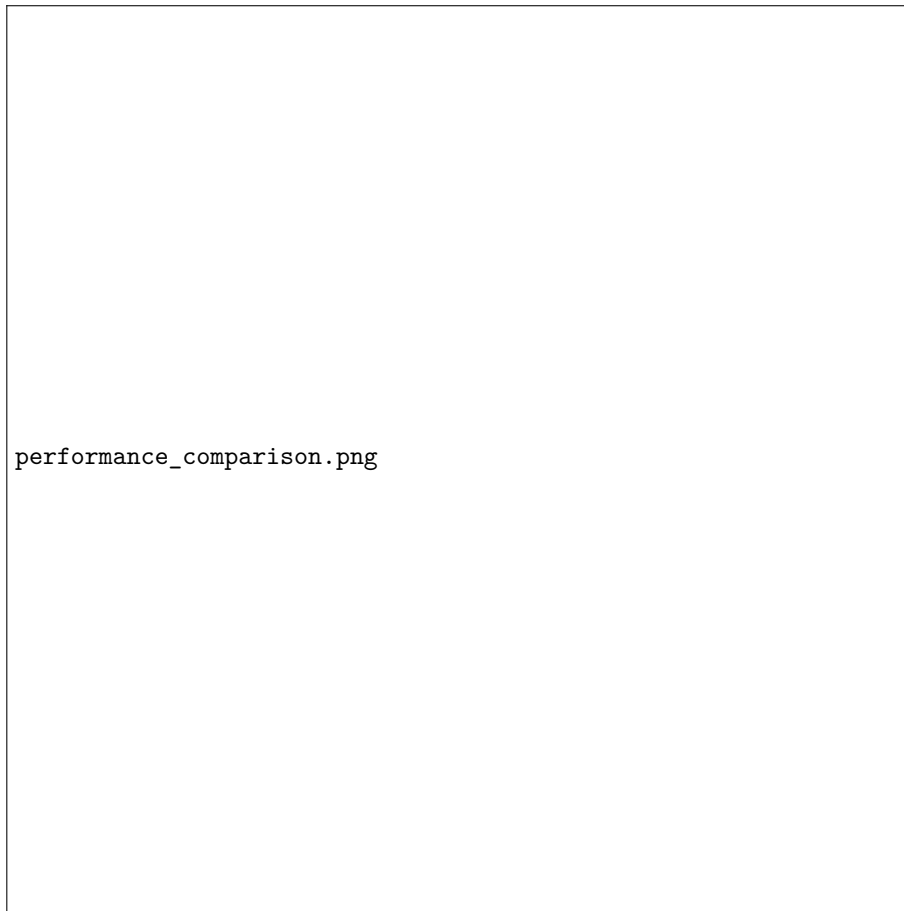


Figure 1: Total solution time comparison across the three configurations. The Mixed Precision (Tuned) approach consistently outperforms the Baseline and Mixed Precision (Fixed) methods.



Figure 2: Relative Error Comparison. The Mixed Precision solver maintains the same order of accuracy (10^{-16}) as the Double Precision baseline, confirming that the speedup does not compromise solution quality.

3.2 Tolerance Sensitivity Analysis

The impact of the inner solver tolerance τ on total solution time was analyzed to try and find an optimal switching point. The data for the `consph` matrix (Figure ??) illustrates the theoretical Descent-to-Ascent behavior:

- **Looser Tolerance** ($\tau = 10^{-1}$): Time = 23.45s. The inner solver is too inaccurate, forcing the expensive outer loop to perform more iterations to correct the error.
- **Optimal Tolerance** ($\tau = 10^{-2}$): Time = 21.04s. This is the sweet spot where the inner solver does enough work to be effective but stops before wasting cycles on converged bits that will be lost during the update.
- **Tighter Tolerance** ($\tau = 10^{-4}$): Time = 33.53s. The inner solver works too hard to achieve precision that is unnecessary for the outer loop update, degrading overall performance.

Similar behavior was observed for `cfd1` (Figure ??), where $\tau = 10^{-2}$ was also optimal. Interestingly, `cfd2` (Figure ??) benefited from a tighter tolerance ($\tau = 10^{-4}$), suggesting that for certain condition numbers, higher precision in the correction step is worth the computational cost.



Figure 3: Sensitivity analysis for the **consph** matrix. The curve exhibits a clear minimum at $\tau = 10^{-2}$, validating the hypothesis that solving to full single precision (e.g., 10^{-7}) is often suboptimal.

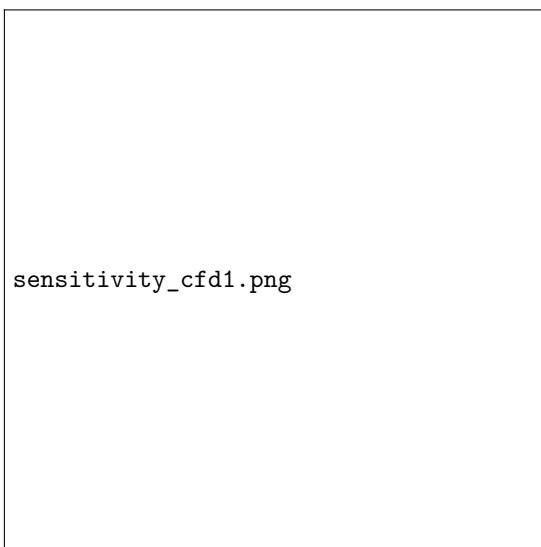


Figure 4: Sensitivity Analysis for **cfd1**.



Figure 5: Sensitivity Analysis for **cfd2**.

4 Conclusion

From this Study of Mixed Precision Iterative refinement systems, we learned that it is a very effective strategy for speeding up the time in solving large sparse symmetric positive definite systems. This is very important for real world hyper computing problems, such as the Pressure Poisson Equation or 2D/3D engineering problems. By offloading most of the computational work to a single-precision inner solver while also maintaining a double-precision outer loop, we were able to test the theory that mixed-precision significantly improves performance gains without compromising the final accuracy of the solution.

The results from our experiment show that the MPIR approach consistently outperformed standard a double-precision Conjugate Gradient solver. The fixed-tolerance configuration ($\tau = 10^{-7}$) showed speedups ranging from $2.1\times$ to $6.0\times$ and, by tuning the inner solver's termination tolerance (τ), we achieved even greater efficiency, with speedups reaching as high as $8.6\times$ for the `cf2` matrix. This confirmed that solving the inner correction system to full single-precision accuracy is often unnecessary and computationally wasteful.

The sensitivity results revealed a "descent-to-ascent" behavior in the total solution time as a function of τ . For most matrices, a looser tolerance (e.g., $\tau = 10^{-2}$) provided the optimal balance between inner solver cost and outer loop convergence. We found though, the optimal τ is problem-dependent, we see with `cf2`, which benefited from a tighter tolerance. This suggests that the strategies for selecting τ could be a valuable direction for future research.

In conclusion, Mixed Precision Iterative Refinement offers a strong path for optimizing solving systems of Algebraic expressions with linear solvers in High Performance Computing. The better we are able to comprehend and optimize this strategy, the more optimized systems we will be able to create.